Unit IV: Software Testing and Maintenance

Testing – Unit testing – Black box testing– White box testing – Integration and System testing– Regression testing – Debugging – Program analysis – Symbolic execution – Model Checking-Case Study

4.1 Testing

- The aim of program testing is to help realize identify all defects in a program. It is not possible to guarantee that a program is error free.
- This is because the input data domain of most programs is very large, and it is not practical to test the program exhaustively with respect to each value that the input can assume.

Basic Concepts and Terminologies

4.1.1Mistake

- A **mistake** is essentially any programmer action that later shows up as an incorrect result during program execution.
- A programmer may commit a mistake in almost any development activity.
- For example, during coding a programmer might commit the mistake of not initializing a certain variable, or might overlook the errors that might arise in some exceptional situations such as division by zero in an arithmetic operation.

4.1.2Error

- An **error** is the result of a mistake committed by a developer in any of the development activities.
- Among the extremely large variety of errors that can exist in a program. One example of an error is a call made to a wrong function.

4.1.3 Failure

- A **failure** of a program essentially denotes an incorrect behavior exhibited by the program during its execution.
- An incorrect behaviour is observed either as an incorrect result produced or as an in appropriate activity carried out by the program.
- Every failure is caused by some bugs present in the program.

4.1.4 Test case

- A **test case** is a triplet [I , S, R], where I is the data input to the program under test, S is the state of the program at which the data is to be input, and R is the result expected to be produced by the program.

4.1.5 Test scenario

- A **test scenario** is an abstract test case in the sense that it only identifies the aspects of the program that are to be tested without identifying the input, state, or output.
- A test case can be said to be an implementation of a test scenario.
- In the test case, the input, output, and the state at which the input would be applied is designed such that the scenario can be executed.

4.1.6 Test script

- A **test script** is an encoding of a test case as a short program. Test scripts are developed for automated execution of the test cases.
- A test case is said to be a **positive test case** if it is designed to test whether the software correctly performs a required functionality.
- A test case is said to be negative test case.If it is designed to test whetherthe software carries out something, that is not required of the system.

4.1.7 Test suite

- A **test suite** is the set of all test that have been designed by a tester to test a given program.

4.1.8 Testability

- **Testability** of a requirement denotes the extent to which it is possibleto determine whether an implementation of the requirement conformsto it in both functionality and performance.

4.1.9Failure mode

- A **failure mode** of software denotes an observable way in which it can fail.

4.1.10 Equivalent faults

- **Equivalent faults** denote two or more bugs that result in the system failing in the same failure mode.

Testing activities

Testing involves performing the following major activities:

- **Test suite design**: The test suite is designed possibly using **several test case design techniques.**
- **Running test cases and checking the results to detect failures**: Each test case is run and the results are compared with the expected results. A mismatch between the actual result
- and expected results indicates a failure. The test cases for which the system fails are noted down for later debugging.
- **Locate error:** In this activity, the failure symptoms are analysed to locate the errors.
- For each failure observed during the previous activity, **the statements that are in error are identified**.
- **Error correction:** After the error is located during debugging**, the code is appropriately changed to correct the error.**

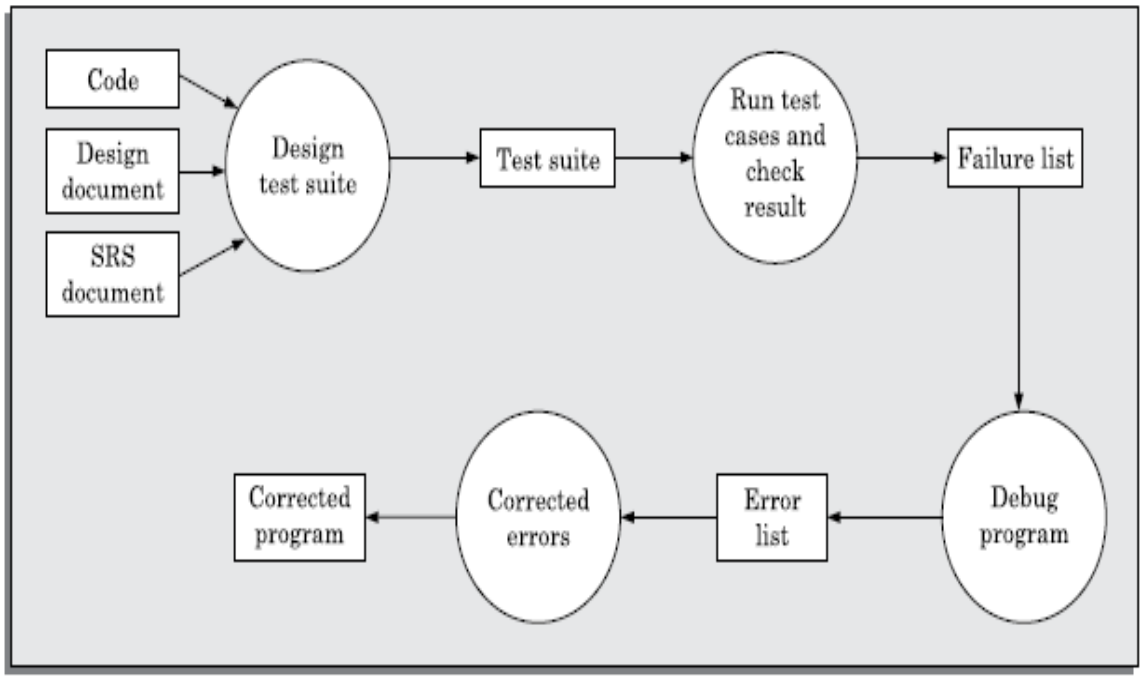


FIGURE 10.2 Testing process.

➢

Verification versus validation

- The objectives of both verification and validation techniques are very similar since both these techniques are designed to help remove errors in software.

Verification

- Verification is the process of determining whether the output of one phase of software development conforms to that of its previous phase;
- The objective of verification is to check if the work products produced after a phase conform to that which was input to the phase.
- For example, a verification step can be to check if the design documents produced after the design step conform to the requirements specification.
- The primary techniques used for verification include review, simulation, formal verification, and testing.
- Verification does not require execution of the software.
- Verification is carried out during the development process to check if the development activities are proceeding alright

Validation

- Validation is the process of determining whether a fully developed software conforms to its requirements specification.
- Validation is applied to the fully developed and integrated software to check if it satisfies the customer's requirements.
- Validation requires execution of the software.

- Validation is carried out to check if the right as required by the customer has beendeveloped.
- The primary objective of the verification steps are to determine whether the steps in product development are being carried out alright, whereas validation is carried out towards the end of the development process to determine whether the right product has been developed.
- While verification is concerned with phase containment of errors, the aim of validation is to check whether the deliverable software is error free.
- Error detection techniques = Verification techniques + Validation techniques

No	Verification	Validation
1	It means "Are we implementing the software right?"	It means "Are we implemented the right software?"
2	It comes before validation	It comes after verification
3	It is known as Static Testing.	It is known as Dynamic Testing.
4	It is executed by Quality assurance team or Developers.	It is executed by the Testing team.
5	It includes checking documents, design, code and program.	It includes testing and validating the actual product
6	Whether the software conforms to specification is checked	It checks whether the software meets the requirements and expectations of a customer
7	It does not involve executing the code	It always involves executing the code

No	Verification	Validation
8	It finds bugs early in the development cycle	It can find bugs that the verification process can not catch
9	It includes different methods like Inspections, Reviews & Walkthroughs.	It includes testing like Functional, System, Integration & User acceptance testing
10	Developer can verify that the inputs follow the outputs or not.	Tester can validate that the user accepts the product or not.

4.2.UNIT TESTING

- Unit testing focuses verification effort on the smallest unit of software design.
- The software component or module.
- Using the component-level design description as aguide, important control paths are tested to uncover errors within the boundary ofthe module.
- The relative complexity of tests and uncovered errors is limited by theconstrained scope established for unit testing.
- The unit test is white-box oriented,and the step can be conducted in parallel for multiple components.
- Unit testing is undertaken after a module has been coded and reviewed.
- This activity is typically undertaken by the coder of the module himself in the coding phase.
- Before carrying out unit testing, the unit test cases have to be designed and the test environment for the unit under test has to be developed.

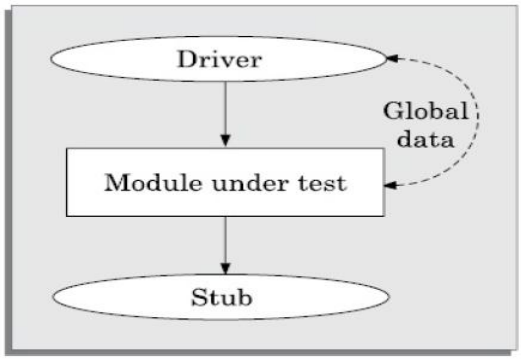
4.2.1 Driver and stub modules

- In order to test a single module, we need a complete environment to provide all relevant code that is necessary for execution of the module.
- That is, besides the module under test, the following are needed to test the module:

- The procedures belonging to other modules that the module under test calls.
- Non-local data structures that the module accesses.
- A procedure to call the functions of the module under test withappropriate parameters.
- Modules required to provide the necessary environment (which either call or are called by the module under test) are usually not available until they too have been unit tested.
- Stubs and drivers are designed to provide the complete environment for a module so that testing can be carried out.

4.2.2 Stub:

- The role of stub and driver modules is pictorially shown in below Figure.
- A stub procedure is a dummy procedure that has the same I/O parameters as the function called by the unit under test but has a highly simplifiedbehaviour.
- For example, a stub procedure may produce the expected behaviour using a simple table look up mechanism.



Unit testing with the help of driver and stub modules.

4.2.3 Driver:

- A driver module should contain the non-local data structures accessed by the module under test.
- Additionally, it should also have the code to call the different functions of the unit under test with appropriate parameter values for testing.

4.2.4 Unit Test Considerations:

- The module interface is tested to ensure that information properly flows into and outof the program unit under test.
- The local data structure is examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution.
- Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing.
- All independent paths (basis paths) through the control structure are exercised to ensure that all statements in a module have been executed at least once and finally all error handling paths are tested.

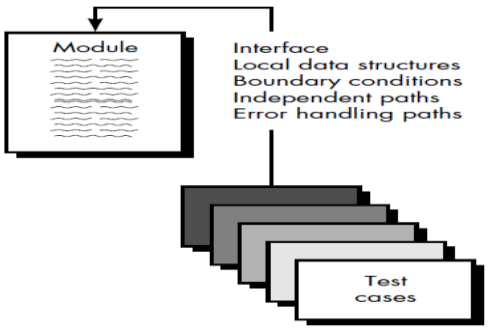


Figure: unit testing

Among the more common errors in computation are

(1) Misunderstood or incorrectarithmetic precedence

(2) Mixed mode operations

(3) Incorrect initialization

(4) Precision inaccuracy

(5) Incorrect symbolic representation of an expression

Test cases should uncover errors such as

(1) Comparison of different data types,

(2) Incorrect logical operators or precedence,

(3) Expectation of equality when precision error makes equality unlikely,

(4) Incorrectcomparison of variables,

(5) Improper or nonexistent loop termination,

(6) Failureto exit when divergent iteration is encountered, and

(7) Improperly modified loopvariables.

Among the potential errors that should be tested when error handling is evaluated are

1. Error description is unintelligible.

2. Error noted does not correspond to error encountered.

3. Error condition causes system intervention prior to error handling.

4. Exception-condition processing is incorrect.

5. Error description does not provide enough information to assist in the location of the cause of the error.

- ❖ Boundary testing is the last (and probably most important) task of the unit test step.
- ❖ Software often fails at its boundaries.
- ❖ That is, errors often occur when the *n*th element of an *n*-dimensional array is processed.
- ❖ When the *i*th repetition of a loop with *i*passes is invoked, when the maximum or minimum allowable value is encountered.

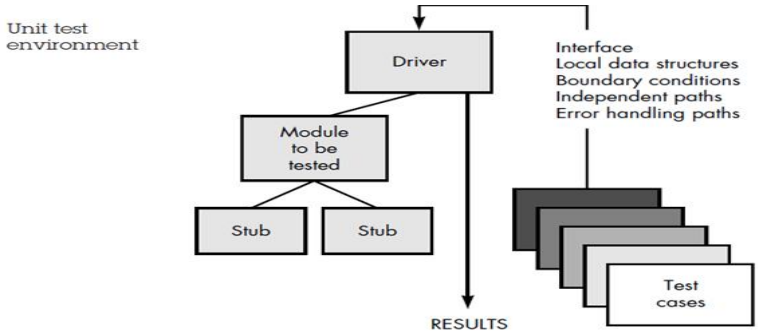


Figure: Unit test Environment

4.3.BLACK-BOX TESTING

- ❖ In black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design or code is required.
- ❖ Black-box testing, also called behavioral testing, focuses on the functional requirementsof the software.

- ❖ Black-box testing enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- ❖ Black-box testing attempts to find errors in the following categories:
  (1) Incorrect or missing functions,
  (2) Interface errors,
  (3) Errors in data structures or external database access,
  (4) Behavior or performance errors, and
  (5) Initialization and termination errors.

The following are the four main approaches available to design black box test cases:

Black box testing

- No knowledge of **internal program design or code required.**
- Tests are based on **requirements and functionality**.

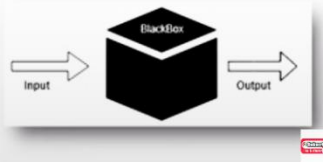
White box testing

- Knowledge of the **internal program design and code required.**
- Tests are based on **coverage of code statements, branches, paths, conditions.**

BLACK-BOX TESTING TECHNIQUES

In black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design or code is required.

- In Black Box Testing, the functionalities of software applications are tested **without having knowledge** of Internal code structure, Implementation details and Internal paths.
- Tester gives **input value** to examine its functionality & checks whether function is giving **expected output or not**.
- If the function produces correct output, then it is passed in testing, otherwise failed.
- It is also known as Behavioral Testing, Functional & Closed box Testing.
- Perform by the Software Testers.



- **Black Box Testing Tools:** QTP, Selenium, Loadrunner & Jmeter

The following are the two main approaches available to design black box test cases:

1. ④ Equivalence class partitioning
2. ④ Boundary value analysis

Equivalence Partitioning

- ➢ In the equivalence class partitioning approach, the domain of input values to the unit under test is partitioned into a set of equivalence classes.

- The partitioning is done such that for every input data belonging to the same equivalence class, the program behaves similarly.
- Equivalence classes for a unit under test can be designed by examining the input data and output data.

.

1. Equivalence Partitioning:

- Here, input values that provide to system are divided into different classes or groups based on its similarity in the outcome.
- Instead of using each & every input value, use any one value from the group to test outcome.

❖ Equivalence classes may be defined according to the following guidelines:

1. If an input condition specifies a range, one valid and two invalid equivalenceclasses are defined.
2. If an input condition requires a specific value, one valid and two invalidequivalence classes are defined.
3. If an input condition specifies a member of a set, one valid and one invalidequivalence class are defined.
4. If an input condition is Boolean, one valid and one invalid class are defined.

AGE Enter Age * Accepts value from 18 to 60

Equivalence Class Partitioning		
Invalid	Valid	Invalid
<=17	18-60	>=61

4.3.3 Boundary Value Analysis

- It test, boundary values are those that contain the upper and lower limit of a variable.
- It tests, while entering boundary value whether the software is producing correct output or not.

AGE Enter Age *Accepts value 18 to 56

- A type of programming error that is frequently committed by programmers ismissing out on the special consideration that should be given to the values atthe boundaries of different equivalence classes of inputs.
- The reason behind programmers committing such errors might purely be due to psychologicalfactors.
- Programmers often fail to properly address the special processing required by the

input values that lie at the boundary of the differentequivalence classes.

- For example, programmers may improperly use <instead of <=, or conversely <= for <, etc.
- For reasons that are not completely clear, a greater number of errors tends to occur at the boundaries of the input domain rather than in the "center."
- It is for this reasonthat boundary value analysis (BVA) has been developed as a testing technique.
- Boundaryvalue analysis leads to a selection of test cases that exercise bounding values.
- Boundary value analysis is a test case design technique that complements equivalencepartitioning.

❖ Guidelines for BVA are similar in many respects to those provided for equivalence partitioning:

1. If an input condition specifies a range bounded by values a and b, test casesshould be designed with values a and b and just above and just below aand b.

2. If an input condition specifies a number of values, test cases should be developedthat exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.

3. Apply guidelines 1 and 2 to output conditions. For example, assume that atemperature vs. pressure table is required as output from an engineeringanalysis program. Test cases should be designed to create an output reportthat produces the maximum (and minimum) allowable number of table entries.

4. If internal program data structures have prescribed boundaries (e.g., an arrayhas a defined limit of 100 entries), be certain to design a test case to exercisethe data structure at its boundary.

BOUNDARY VALUE ANALYSIS		
Invalid (min -1)	Valid (min, +min, -max, max)	Invalid (max +1)
17	18, 19, 55, 56	57

3. Decision Table Testing:

- Various input combinations & their respective system behavior are captured in tabular form.
- Check logical relationship between two and more than two inputs. **Ex.** Gmail Account

Email (condition1)	T	T	F	F
Password (condition2)	T	F	T	F
Expected Result (Action)	Account Page	Incorrect password	Incorrect email	Incorrect email

4. Error Guessing:

- It is based on the experience of the tester, where tester uses experience to guess the problematic areas of the software.
- **Examples:** Divide by zero, Handling null values in text fields, Accepting the Submit button without any value, File upload without attachment, File upload with less than or more than the limit size.

➢Advantages of Black Box Testing:

1. The tester does not need to have more functional knowledge or programming skills.
2. It is efficient for implementing the tests in the larger system.
3. Tests are executed from the user's or client's point of view.
4. It is used in finding the ambiguity and contradictions in the functional specifications.

➢Disadvantages of Black Box Testing:

1. Without clear programming knowledge, test cases are difficult to implement.
2. It does not reveal the errors in the control structure.
3. Working with a large sample space of inputs can be exhaustive and consumes a lot of time.

4.4. WHITE-BOX TESTING

- White-box testing, sometimes called *glass-box testing* is a test case design method that uses the control structure of the procedural design to derive test cases.

- White box testing techniques analyze Internal design, Code structure & working of the complete software product.
- It is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.
- Tester has knowledge about the internal structure or the programming of the software.
- It is mostly done by software developers.
- Perform with Unit Testing & Integration Testing.
- **White Box Testing Tools:** EclEmma, Nunit, PyUnit, HTMLUnit, CppUnit

- Using white-box testing methods, the software engineer can derive test cases.

(1) Guaranteethat all independent paths within a module have been exercised at least once.
(2) Exercise all logical decisions on their true and false sides.
(3) Execute all loops attheir boundaries and within their operational bounds.
(4) Exercise internal datastructures to ensure their validity.

BASIS PATH TESTING

- The basis path method enables the test case designer to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths.
- Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.

1.Path Coverage

A test suite achieves path coverage if it exeutes each linearly independent paths (or basis paths) at least once.

- Testing is dependent on the program's control or flow structure.
- All possible paths are defined with entry to exit points in the system.

- A linearly independent path can be defined in terms of the control flow graph (CFG) of a program.
- Therefore, to understand path coverage-based testing strategy,we need to first understand how the CFG of a program can be drawn**.**

Control flow graph (CFG)

- A control flow graph describes how the control flowsthrough the program.

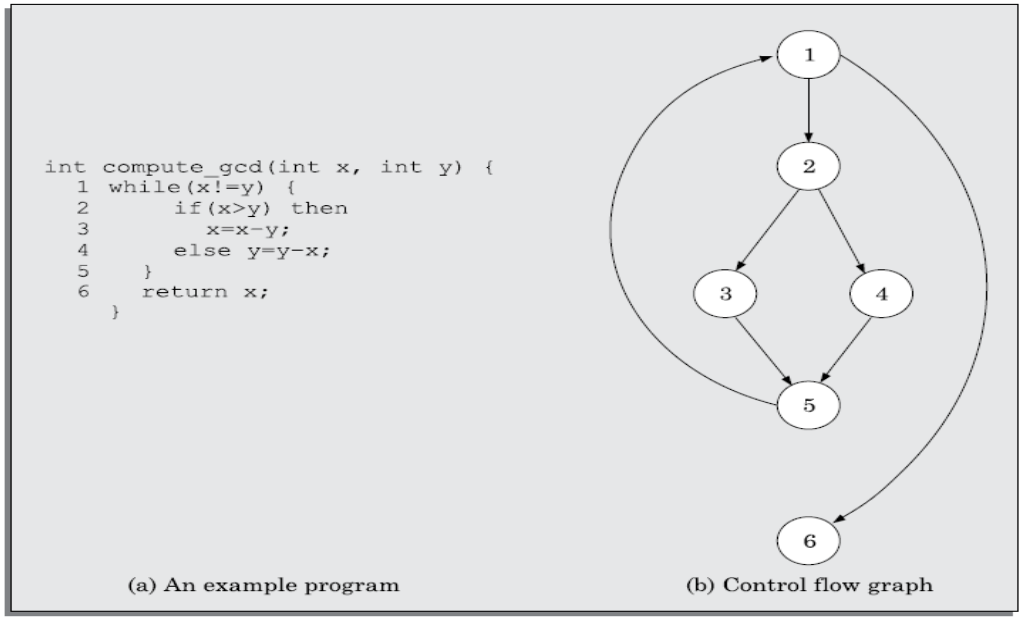


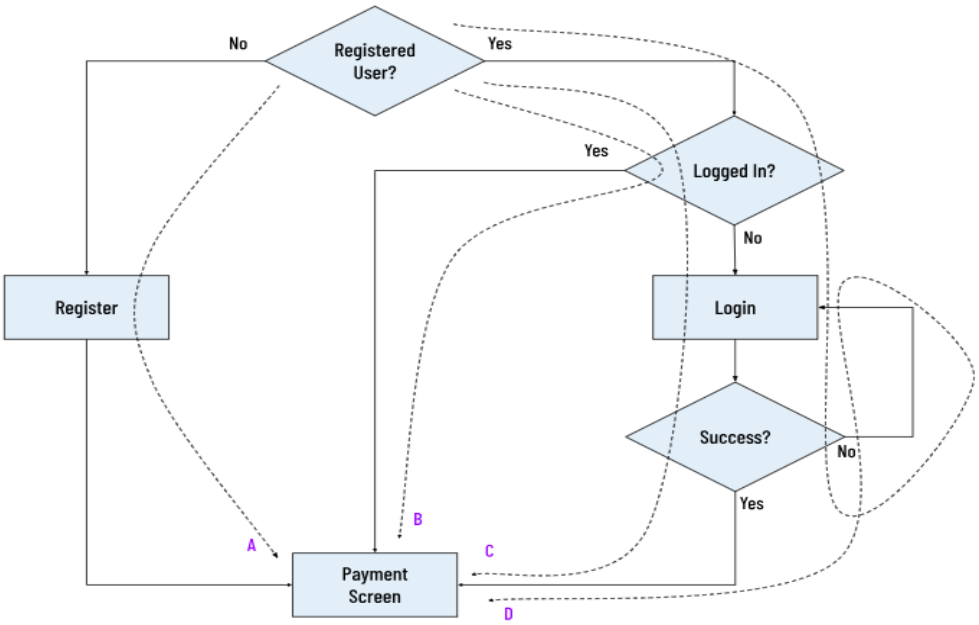
FIGURE 10.8 Control flow diagram of an example program.

Path

- A path through a program is any node and edge sequence from the start node to a terminalnode of the control flow graph of a program.
- **Linearly independent set of paths (or basis path set)**

A set of paths for a given program is called linearly independent set of paths (or the set ofbasis paths or simply the basis set), if each path in theset introduces at least one new edge that is not includedin any other path in the set.

Example: The test cases in this technique ensure that every complete path in the program has been executed at least once. In the flow-chart representation of the code in the example below, the four arrows represent the test cases to cover all paths.



Loop TESTING

How to do Loop Testing: Complete Methodology

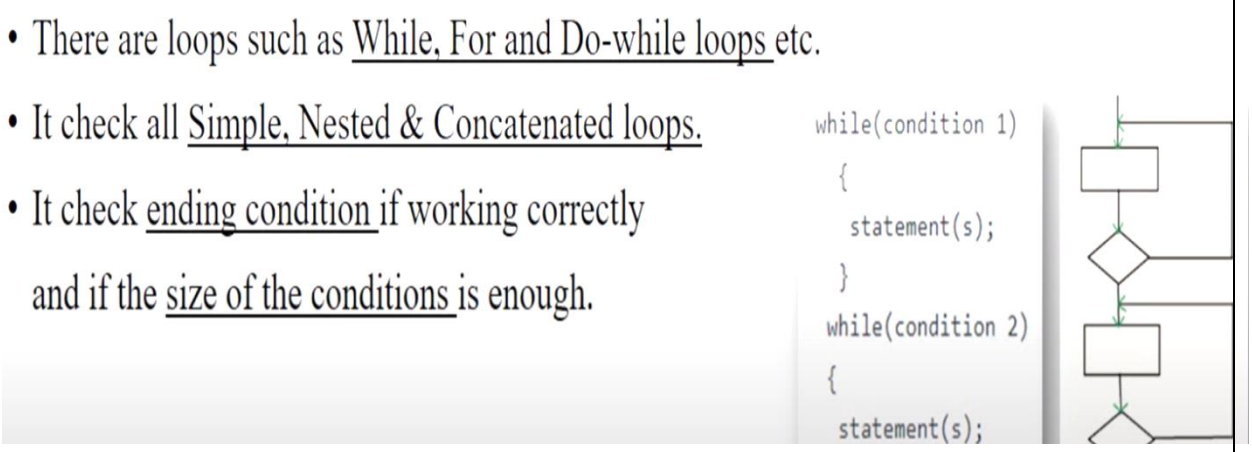
While testing loop, it has to be checked at three different levels:

1. When loop is entered
2. During its execution and
3. When the loop is left

Simple loop

A simple loop is tested in the following way:

1. Skip the entire loop
2. Make 1 passes through the loop
3. Make 2 passes through the loop
4. Make a passes through the loop where a<b, n is the maximum number of passes through the loop



3.Branch Coverage

Branch coverage is also called decision coverage (DC).

- It is also sometimes referred to asall edge coverage.
- A test suite achieves branch coverage, if it makes the decision expression

in each branch in the program to assume both true and false values.

- In other words, forbranch coverage each branch in the CFG representation of the program must be taken atleast once, when the test suite is executed.
- Branch testing is also known as all edge testing,since in this testing scheme, each edge of a program's control flow graph is required tobe traversed at least once.

Example1

Condition Coverage technique

- It is used to cover all conditions.
- Condition coverage is also known as Predicate Coverage in which each one of the Boolean expression have been evaluated to both TRUE and FALSE.

```
read a, b, c;
if(a==0|| b==0)
{
   print 1;
}
else
{
   if(c==0 && d==0)
   {
     print 2;
   }
}
```

Test cases should be designed to evaluate every conditions

Testcase1:a=0,b=0
Testcase2:a=1,b=0 c=0,d=0
Testcase3:a=1,b=1c=0,d=0

Example2

```
function doSomething(int age, boolean hasParentalConsent)
{
    if (age < 18 AND hasParentalConsent == false)
    {
        error(' Must be an adult or parental consent is required');
    }
    ...
}
```

In this example, you will write the following test cases to cover the evaluation of the expression: `age < 18 and hasParentalConsent == false`:

- age is 17, hasParentalConsent is false
- age is 17, hasParentalConsent is true
- age is 18, hasParentalConsent is false
- age is 18, hasParentalConsent is true
- age is 19, hasParentalConsent is false
- age is 19, hasParentalConsent is true

4.Statement Coverage

Statement coverage is a metric to measure the percentageof statements that are executed by a test suite in a programat least once.

- Statement coverage is a testing technique in which tester traverse all statements in code.
- It ensure that each statement of the code is executed at least once.
- Hence each line of the code is verified

Design a statement coverage-based test suite for the following Euclid's GCD computation function:

```
int computeGCD(int x,int y){
        1 while (x != y){
        2 if (x>y) then
        3 x=x-y;
        4 else y=y-x;
        5 }
        6 return x;
        }
```

Solution:

- To design the test cases for achieving statement coverage, the conditional

expression of the while statement needs to be made true and the conditional expressionof the if statement needs to be made both true and false.

- By choosing the test set {(x =3, y = 3), (x = 4, y = 3), (x = 3, y = 4)}, all statements of the program would be executedat least once.

Example 2:

```
print (int a, int b) {
int sum = a+b;
if (sum>0)
print ("This is a positive result")
else
print ("This is negative result")
}
```

Scenario 1:
If a = 5, b = 4

Total number of statements = 7
Number of executed statements = 5

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

Statement coverage = 5/7*100
= 500/7
= 71%

➢Advantages of White Box Testing:

1. White box testing optimizes code so hidden errors can be identified.
2. Test cases of white box testing can be easily automated.
3. Early detection of errors that improve code quality.
4. It cover most path & code of the software.

➢Disadvantages of White Box Testing:

1. White box testing can be quite Complex, Expensive & Time-consuming.
2. Requires professional resources with detailed understanding of programming.
3. Missing functionalities cannot be detected.
4. Redesign of code needs test cases to be written again.

Testing Levels

A software product is normally tested in three levels or stages:

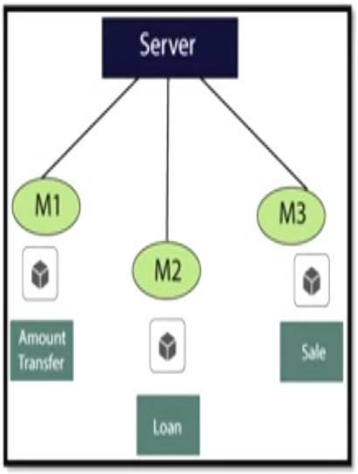
1. Unit testing
2. Integration testing
3. System testing

- During unit testing, **the individual functions (or units) of a program are tested**.
- After testing **all the units individually, the units are incrementally integrated and tested after each step of integration** (integration testing).
- Finally, the fully integrated system is tested (system testing). Integration and system testing are usually referred to as *testing in the large.*
- Unit testing is referred to as testing in the small, where as integration and system testing are referred to as *testing in the large.*

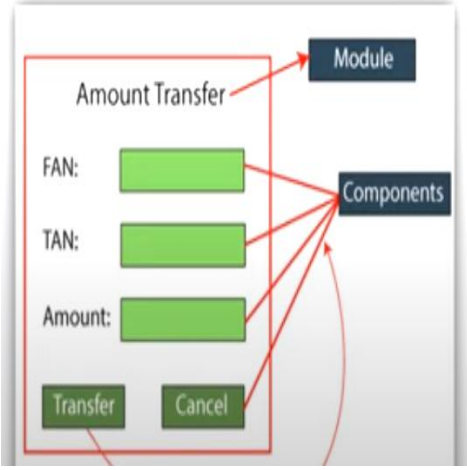
1. Unit testing

- Unit testing is the first level of testing done before other remaining levels of the testing.
- In Unit Testing individual Units or Components of a software are tested.
- A Unit may be an individual function, method, procedure, module or object in program.
- Every module of project check one by one which is also called as components testing.
- It used white box testing techniques.
- It is done during coding phase of SDLC by the Software Developers.
- **Unit Testing Tools:** JUnit, NUnit, PHPUnit, EMMA & Jmockit

Example:



For From Account Number (FAN) & To Account Number (TAN)	
Values	**Description**
1234	Account Number Accepted
4300	Error: Account Number Invalid
Blank	Error: Enter Account Number
Alphanumeric	Error: Enter digits only
Block Acc. No	Error Message



For Transfer Button
Description
Enter valid FAN value
Enter valid TAN value
Enter correct amount
Click on Transfer Button

For Cancel Button
Description
Enter value of FAN, TAN & Amount
Click on Cancel Button: All data should be clear

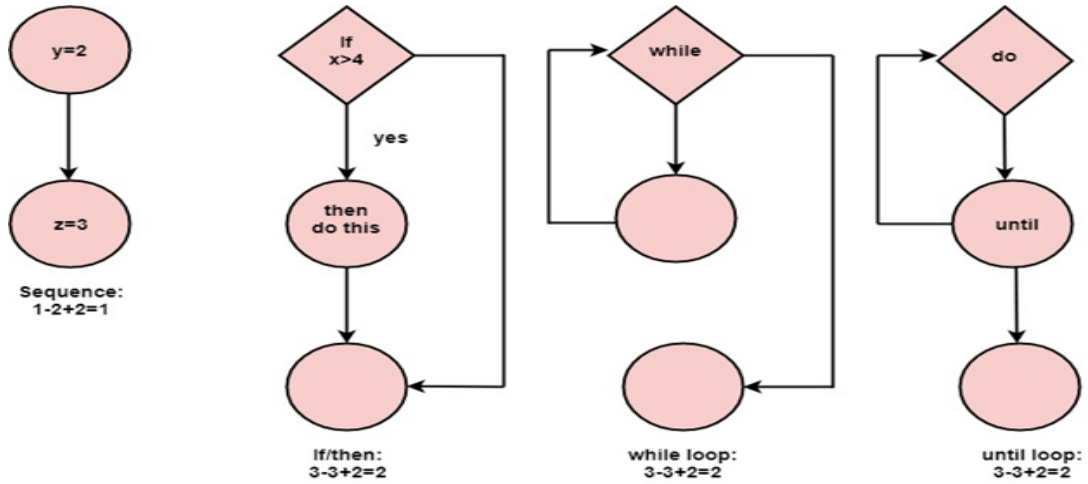
4.4.2. Cyclomatic Complexity

- Cyclomatic complexity is a software metric used to measure the complexity of a program.
- Thomas J. McCabe developed this metric in 1976.McCabe interprets a computer program as a set of a strongly connected directed graph.
- Nodes represent parts of the source code having no branches and arcs represent possible control flow transfers during program execution.

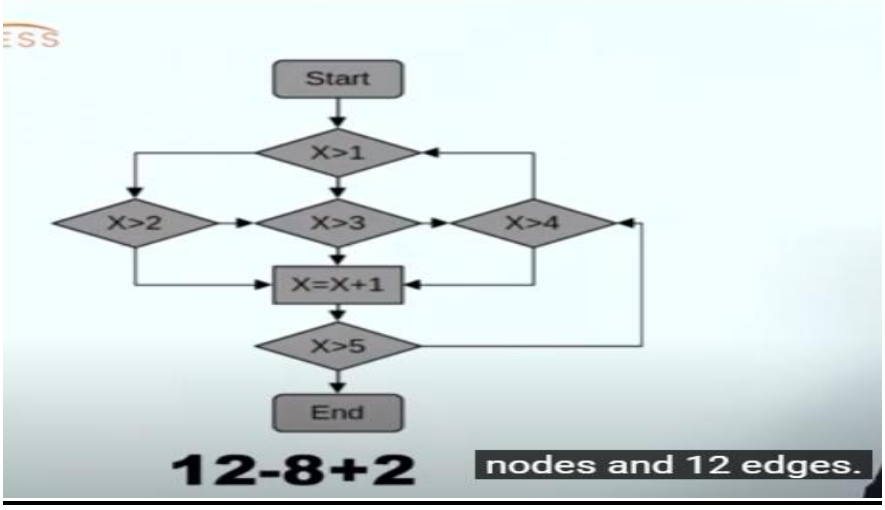
How to Calculate Cyclomatic Complexity?

- McCabe proposed the cyclomatic number, V (G) of graph theory as an indicator of software complexity.
- The cyclomatic number is equal to the number of linearly independent paths through a program in its graphs representation.
- For a program control graph G, cyclomatic number, V (G), is given as:
  V (G) = E - N + 2 * P

E = The number of edges in graphs G
N = The number of nodes in graphs G
P = The number of connected components in graph G.
Example1:



Example2:



4.4.3. Deriving Test Cases

- The basis path testing method can be applied to a procedural design or to sourcecode.
- The following steps canbe applied to derive the basis set:
  1. Using the design or code as a foundation, draw a corresponding flowgraph.
  2. Determine the cyclomatic complexity of the resultant flow graph.
  3. Determine a basis set of linearly independent paths.
  4. Prepare test cases that will force execution of each path in the basisset.

4.5. INTEGRATION TESTING

- Integration testing is carried out after all (or at least some of) the modules have been unit tested.
- Successful completion of unit testing, to a large extent, ensures that the unit (or module) as a whole works satisfactorily.
- In this context, the objective of integration testing is to detect the errors at the module interfaces (call parameters).
- For example, it is checked that no parameter mismatch occurs when one module invokes the functionality of another module.
- Thus, the primary objective of integration testing is to test the module interfaces, i.e., there are no errors in parameter passing, when one module invokes the functionality of another module.
- Integration testing is a systematic technique for constructing the program structurewhile at the same time conducting tests to uncover errors associated with interfacing.
- The objective is to take unit tested components and build a program structurethat has been dictated by design.

- Integration testing is the second level of the software testing process comes after unit testing
- All modules may be coded by the different programmers or developers.
- Integrating various components or modules of an application & testing their behavior as a single unit or group called as Integration Testing.
- Also known as Thread testing or String testing
- The goal of Integration Testing is to check correctness & communication among all modules.

➢ **Integration Testing Tools:**

Selenium, PyTest, Junit, Jasmine, Steam, Mockito etc.

Example

After User 1 Login to Gmail Account

User 1: Click on Compose mail, Fill details & Send Mail to User 2.

If not send mail save in 'Draft' otherwise save in 'Sent Items'.

Log out the Account.

After User 2 Login to Gmail Account

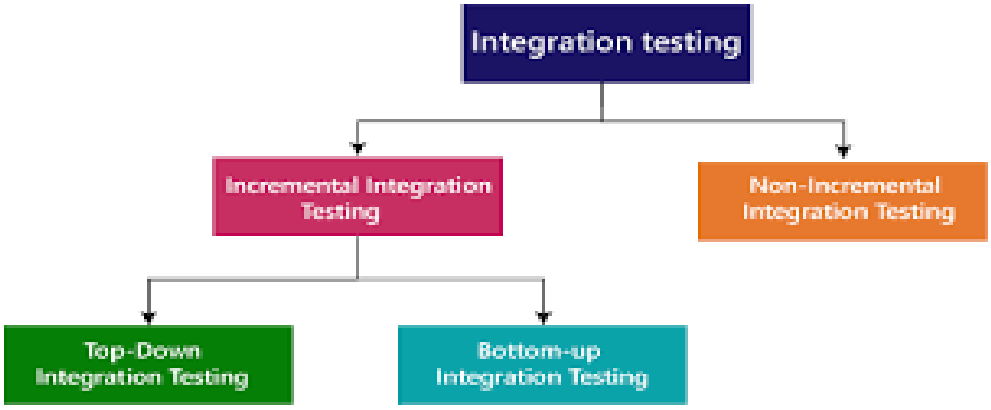
User 2: Move to the 'Inbox' and verify that if the mail has reached.

Open mail & check.

If needed, Reply it.

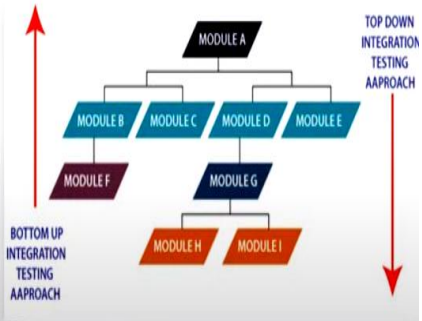
Log out the Account.

Types of Integration Testing



1. Incremental Integration Testing:

- Here, modules are added in ascending order one by one or according to need.
- The selected modules must be logically related.
- They test data flow between modules & correctness of each functions.
- The process continues until the successful testing of all the modules.
- **It has two types:** Top-Down & Bottom-Up



Example:

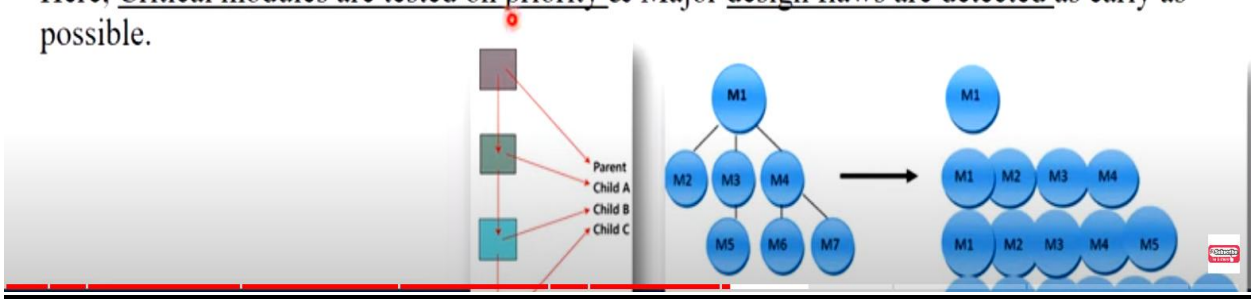
Flipkart Application, Flow of testing application would like this:

❑Flipkart→ Login→ Home → Search→ Add cart→ Payment → Logout

4.5.1 Top-down Integration

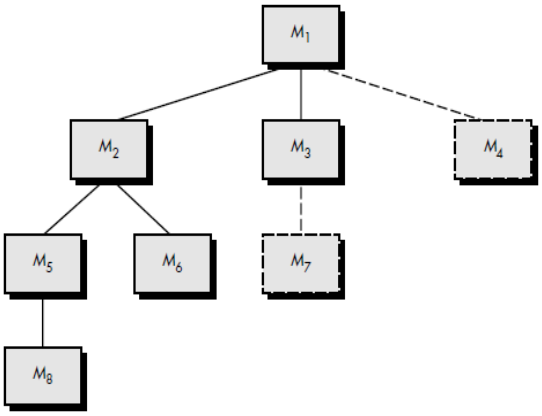
1.1 Top-Down Integration Testing:

- It starts by testing top-most modules & moving down to the lowest set of modules one-by-one & check control flow of system.
- As there is a possibility that the lower level modules might not have been developed while top modules are tested.
- It will use Stubs as a dummy programs which act as a substitutes for the missing models in the testing.
- Here, Critical modules are tested on priority & Major design flaws are detected as early as possible.



- Top-down integration testing is an incremental approach to construction of programstructure.
- Modules are integrated by moving downward through the control hierarchy,beginning with the main control module (main program).
- Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

FIGURE 18.6
Top-down integration



- Referring to Figure 18.6, depth-first integration would integrate all components on a major control path of the structure.
- Selection of a major path is somewhat arbitrary and depends on application-specific characteristics.
- For example, selecting the left handpath, components M1, M2 , M5 would be integrated first. Next, M8 or (if necessary for proper functioning of M2) M6 would be integrated.

- The central and right hand control paths are built. Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally.
- From the figure, components M2, M3, and M4 (a replacement for stub S4) would be integrated first. The next control level, M5, M6, and so on, follows.

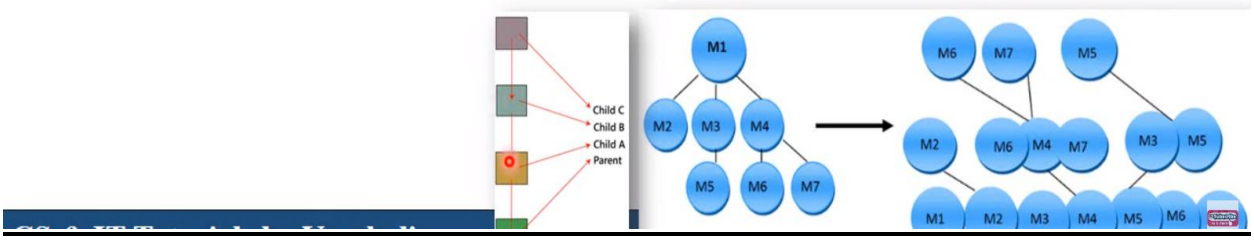
✓ The integration process is performed in a series of five steps:

1. The main control module is used as a test driver and stubs are substituted forall components directly subordinate to the main control module.
2. Depending on the integration approach selected (i.e., depth or breadth first),subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the realcomponent.
5. Regression testing (Section 18.4.3) may be conducted to ensure that newerrors have not been introduced.

4.5.2 Bottom-up Integration

1.2 Bottom-Top Integration Testing:

- It starts with testing the lowest units of the application and moving up one-by-one.
- Here testing takes place from the bottom of the control flow to upwards.
- Again it's possible that the higher level modules might not have been developed by the time lower modules are tested.
- It will use Driver as a dummy programs which act as a substitutes for the missing models in the testing.
- Here, several disjoint subsystems can be tested simultaneously & easy to observe test result.



- Bottom-up integration testing, as its name implies, begins construction and testingwith atomic modules (i.e., components at the lowest levels in the program structure).
- Because components are integrated from the bottom up, processing required for components subordinate to a given level is always available and the need for stubs iseliminated.

✓ A bottom-up integration strategy may be implemented with the following steps:

1. Low-level components are combined into clusters (sometimes called builds)that

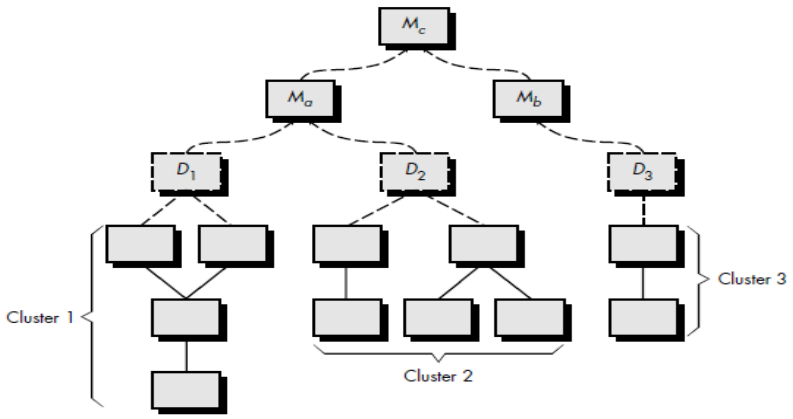
perform a specific software subfunction.

2. A driver (a control program for testing) is written to coordinate test caseinput and output.

3. The cluster is tested.

4. Drivers are removed and clusters are combined moving upward in the programstructure.

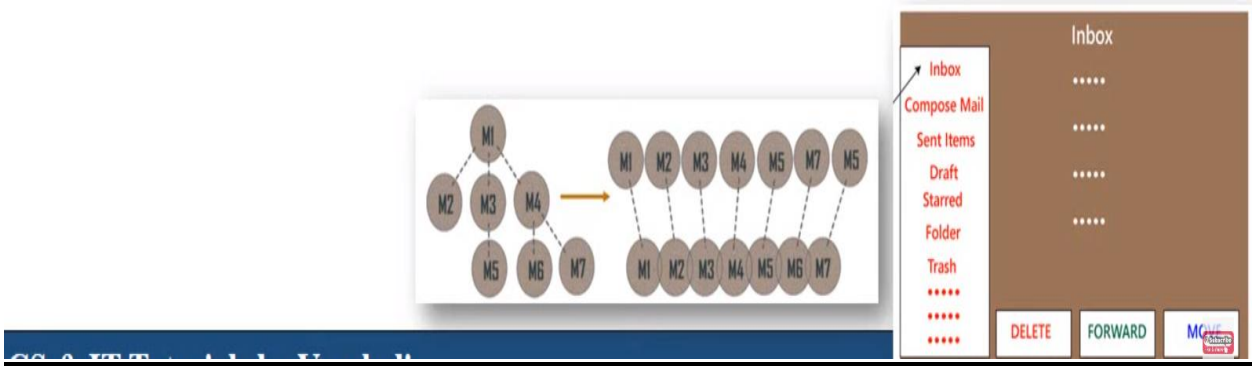
FIGURE 18.7
Bottom-up integration



- Integration follows the pattern illustrated in Figure 18.7.
- Components are combinedto form clusters 1, 2, and 3.
- Each of the clusters is tested using a driver (shown as a dashed block).
- Components in clusters 1 and 2 are subordinate to Ma.
- Drivers D1 and D2 are removed and the clusters are interfaced directly to Ma. Similarly, driver D3 for cluster 3 is removed prior to integration with module Mb.
- Both Ma and Mb will ultimately be integrated with component Mc, and so forth.

2. Non - Incremental Integration Testing / Big Bang Testing

- In this, once all the modules are developed and tested individually, they are integrated once and tested together at once. No parent & child concept.
- The only advantage of this type of testing is that it is very much suitable for smaller systems.
- If an error is found then it is very difficult to solve because it belong to any of the modules being integrated.
- The goal is to verify the overall functionality of the system and to identify any integration problems that arise when all components are combined.



Stubs	Drivers
A section of **code that imitates the called function** is known as Stubs.	A section of code **that imitates the calling function** is known as Drivers.
The stubs are **developed during the Top-down approach** of incremental integration testing.	The drivers are developed during **the bottom-up approach of incremental integration testing.**
Stubs are developed **when high-level modules are tested, and lower-level modules are not formed.**	Drivers are acquired when **lower-level modules are tested, and higher-level modules are not yet developed.**
The stubs signify **the low-level modules.**	The drivers signify the **high-level modules.**
Fundamentally, the Stubs **are also known as a called program** and initially used in the Top-down integration testing.	The Drivers **are also known as the calling program** and are mainly used in bottom-up integration testing.

4.5.4 Smoke Testing

- Smoke testing is an integration testing approach that is commonly used when “shrink wrapped”
- Software products are being developed. It is designed as a pacing mechanism for time-critical projects, allowing the software team to assess its project on afrequent basis.

✓ In essence, the smoke testing approach encompasses the following activities:

1. Software components that have been translated into code are integrated intoa “build.” A build includes all data files, libraries, reusable modules, and engineeredcomponents that are required to implement one or more productfunctions.
2. A series of tests is designed to expose errors that will keep the build fromproperly performing its function. The intent should be to uncover “show stopper”errors that have the highest likelihood of throwing the software projectbehind schedule.
3. The build is integrated with other builds and the entire product (in its currentform) is smoke tested daily. The integration approach may be top down orbottom up.

- Smoke testing provides a number of benefits when it is applied on complex, timecritical software engineering projects:
  - Integration risk is minimized
  - The quality of the end-product is improved
  - Error diagnosis and correction are simplified
  - Progress is easier to assess.

4.5.5 Comments on Integration Testing

❖ As integration testing is conducted, the tester should identify critical modules. A critical module has one or more of the following characteristics:

(1) Addresses severalsoftware requirements,

(2) Has a high level of control (resides relatively high inthe program structure),

(3) Is complex or error prone (cyclomatic complexity may beused as an indicator), or

(4) Has definite performance requirements. Critical modulesshould be tested as early as is possible. In addition, regression tests should focus oncritical module function.

4.5.6 Integration Test Documentation

- An overall plan for integration of the software and a description of specific tests are documented in a Test Specification.
- This document contains a test plan, and a test procedure,is a work product of the software process, and becomes part of the softwareconfiguration.
- The test plan describes the overall strategy for integration.
- Testing is divided into phases and builds that address specific functional and behavioral characteristics of the software.
- For example, integration testing for a CAD system might be divided into the following test phases:
  - User interaction (command selection, drawing creation, display representation,error

processing and representation).

• Data manipulation and analysis (symbol creation, dimensioning; rotation,computation of physical properties).

• Display processing and generation (two-dimensional displays, threedimensionaldisplays, graphs and charts).

• Database management (access, update, integrity, performance).

- ✓ The following criteria and corresponding tests are applied for all test phases:
  - ❖ **Interface integrity.**

    Internal and external interfaces are tested as eachmodule (or cluster) is incorporated into the structure.
  - ❖ **Functional validity.**

    Tests designed to uncover functional errors are conducted.
  - ❖ **Information content.**

    Tests designed to uncover errors associated with local or global data structures are conducted.
  - ❖ **Performance.**

    Tests designed to verify performance bounds established during software design are conducted.

4.6. SYSTEM TESTING

- ➢ System testing is actually a series of different tests whose primary purpose is tofully exercise the computer-based system.
- ➢ Although each test has a different purpose, all work to verify that system elements have been properly integrated and perform allocated functions.

4.6.1 Recovery Testing

- ➢ Recovery testing is a system test that forces the software to fail in a variety of waysand verifies that recovery is properly performed.
- ➢ If recovery is automatic (performed by the system itself), re-initialization, check pointing mechanisms, data recovery, and restart are evaluated for correctness.

4.6.2 Security Testing

- ➢ Security testing attempts to verify that protection mechanisms built into a systemwill, in fact, protect it from improper penetration.
- ➢ "The system's security must, of course, be tested for invulnerability from frontal attack—butmust also be tested for invulnerability from flank or rear attack."

4.6.3Stress Testing

Stress testing executes a system in a manner that demands resources in abnormalquantity, frequency, or volume. For example,

(1) Special tests may be designed thatgenerate ten interrupts per second, when one or two is the average rate.

(2) Inputdata rates may be increased by an order of magnitude to determine how input functions will respond.

(3) Test cases that require maximum memory or other resources are executed.

(4) Test cases that may cause thrashing in a virtual operating system are designed.

(5) Test cases that may cause excessive hunting for disk-resident dataare created. Essentially, the tester attempts to break the program.

- A variation of stress testing is a technique called sensitivity testing.
- In some situations(the most common occur in mathematical algorithms), a very small range of data contained within the bounds of valid data for a program may cause extreme andeven erroneous processing or profound performance degradation.
- Sensitivity testing attempt to uncover data combinations within valid input classes that may causeinstability or improper processing.

4.6.4 Performance testing

- It is designed to test the run-time performance of software within the context of an integrated system.
- Performance testing occurs throughout all steps in the testingprocess.
- The performance of an individual module may be assessed as white-box tests are conducted.
- Performance tests are often coupled with stress testing and usually require bothhardware and software instrumentation. That is, it is often necessary to measureresource utilization (e.g., processor cycles) in an exacting fashion.

4.7. Regression Testing

- Each time a new module is added as part of integration testing, the software changes.
- New data flow paths are established, new I/O may occur, and new control logic is invoked.
- These changes may cause problems with functions that previously worked flawlessly.
- In the context of an integration test strategy, regression testing is the reexecutionof some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.
- In a broader context, successful tests (of any kind) result in the discovery of errors,and errors must be corrected.
- Whenever software is corrected, some aspect of the software configuration (the program, its documentation, or the data that support it) is changed.
- Regression testing is the activity that helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.
- Regression testing may be conducted manually, by re-executing a subset of all test cases or using automated capture/playback tools.
- Capture/playback tools enable thesoftware engineer to capture test cases and results for subsequent playback and comparison.

✓ The regression test suite (the subset of tests to be executed) contains three different classes of test cases:

• A representative sample of tests that will exercise all software functions.

• Additional tests that focus on software functions that are likely to be affected by the change.
• Tests that focus on the software components that have been changed.

Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.

As shown in Figure 10.9, some
test cases may no more be valid after the change. These have been shown as invalid test case. The rest are redundant test cases, which check those parts of the program code that are not at all affected by the change.

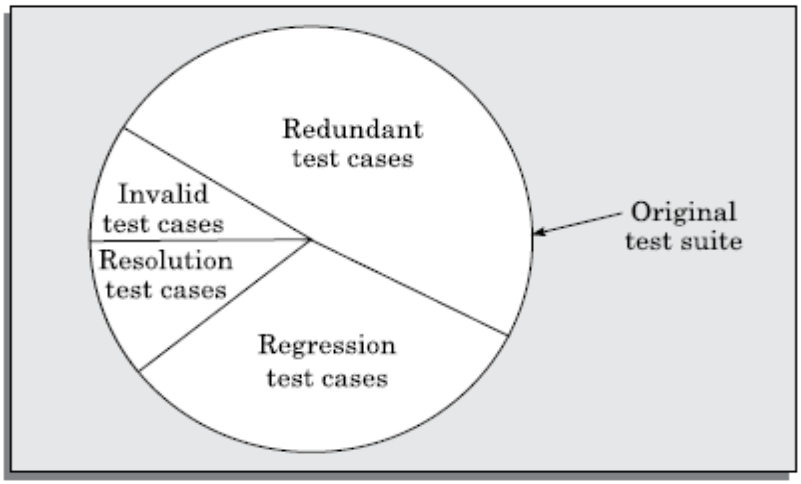


FIGURE 10.9 Types of test cases in the original test suite after a change.

Test automation

- Regression testing after ever change or error correction requires running of several old test cases.
- In this situation, test automation simplifies running of the test cases again and again. Testing tools hold out the promise of substantial cost and time reduction even in the testing and maintenance phases.

It is possible to classify **the tools into the following types** based on the specific methodology on which they are based.

Capture and playback:

- In this type of tools, the test cases are executed manually only once.
- During the manual execution, the sequence and values of various inputs

as well as the outputs produced are recorded.

- On any subsequent occasion, the test can be automatically replayed and the results checked against the recorded output.
- An important advantage of the capture playback tools is that once test data are captured and the results verified, the tests can be rerun easily and cheaply a large number of times.
- Thus, these tools are very useful for regression testing.

- However, capture and playback tools have a few disadvantages as well. Test maintenance can be costly when the unit under test changes since some of the captured tests may become invalid.
- It would require considerable effort to determine and remove the invalid test cases or modify the test input and output data. Also new test cases would have to be added for the altered code.

Test script:

- Test scripts are used to drive an automated test tool. The scripts provide input

to the unit under test and record the output.

- The testers employ a variety of languages to express test scripts.
- An important advantage of test script-based tools is that once the test script is debugged and verified, it can be rerun easily and cheaply a large number of times.
  - However, debugging the test script to ensure its accuracy requires significant effort.
- Also, every subsequent change to the unit under test entails effort to identity impacted test scripts, modify them, rerun and reconfirm them.

Random input test:

- In this type of automatic testing tool, test values are randomly generated cover the input space of the unit under test.
- The outputs are ignored because analyzing them would be extremely expensive.
- The goal is usually to crash the unit under test and not to check if the produced results are correct.
- An advantage of random input testing tools is that it is relatively easy.
- This approach however can be the most cost-effective for finding some types of defects.
- However, random input testing is a very limited form of testing.
- It finds only the defects that crash the unit under test and not the majority of defects that do not crash the system, but simply produce incorrect results.

Model-based test: A model is a simplified representation of program. There can be several types of models of a program. These models can be either structural models or behavioral models. Examples of behavioral models are state models and activity models. A state model-based testing generates tests that adequately covers the state space described by the model.

4.8. DEBUGGING

- After a failure has been detected, it is necessary to first identify the program statement(s) that are in error and are responsible for the failure, the error can then be fixed and the important approaches that are available to identify the error locations.
- Each of these approaches has its own advantages and disadvantages and therefore each will be useful in appropriate circumstances.

Debugging Approaches

The following are some of the approaches that are popularly adopted bythe programmers for

debugging:

4.7.1 Brute force method

- This is the most common method of debugging but is the least efficientmethod.
- In this approach, print statements are inserted throughout the program to print the intermediate values with the hope that some ofthe printed values will help to identify the statement in error.
- Thisapproach becomes more systematic with the use of a symbolic debugger (also called a source code debugger), because values ofdifferent variables can be easily checked and break points and watchpoints can be easily set to test the values of variables effortlessly.

4.7.2 Backtracking

- This is also a fairly common approach. In this approach, starting from thestatement at which an error symptom has been observed, the sourcecode is traced backwards until the error is discovered.
- Unfortunately, as the number of source lines to be traced back increases, the number ofpotential backward paths increases and may become unmanageablylarge for complex programs, limiting the use of this approach.

4.7.3 Cause elimination method

- In this approach, once a failure is observed, the symptoms of the failure (i.e., certain variable is having a negative value though it should bepositive, etc.) are noted.
- Based on the failure symptoms, the causes which could possibly have contributed to the symptom is developed andtests are conducted to eliminate each.
- A related technique ofidentification of the error from the error symptom is the software faulttree analysis.

4.7.4 Program slicing

- This technique is similar to back tracking. In the backtracking approach,one often has to examine a large number of statements. However, thesearch space is reduced by defining slices.
- A slice of a program for aparticular variable and at a particular statement is the set of sourcelines preceding this statement that can influence the value of thatvariable.
- Program slicing makes use of the fact that anerror in the value of a variable can be caused by the statements onwhich it is data dependent.

<u>4.7.5 Debugging Guidelines</u>

- Debugging is often carried out by programmers based on their ingenuity and experience. The following are some general guidelines for effectivedebugging:
- Many times debugging requires a thorough understanding of the program design. Trying to debug based on a partial understanding ofthe program design may require an inordinate amount of effort to beput into debugging even for simple problems.
- Debugging may sometimes even require full redesign of the system. Insuch cases, a common mistake that novice programmers often makeis attempting not to fix the error but its symptoms.

- One must be beware of the possibility that an error correction mayintroduce new errors. Therefore after every round of error-fixing,regression testing must be carried out.

4.9 PROGRAM ANALYSIS TOOLS

- A program analysis tool usually is an automated tool that takes eitherthe source code or the executable code of a program as input andproduces reports regarding several important characteristics of theprogram, such as its size, complexity, adequacy of commenting,adherence to programming standards, adequacy of testing, etc.
- We canclassify various program analysis tools into the following two broadcategories:
  4.8.1 Static analysis tools
  4.8.2 Dynamic analysis tools

4. 9.1 Static Analysis Tools

- Static program analysis tools assess and compute various characteristicsof a program without executing it.
- static analysis tools analyse the source code to compute certain metrics characterising the source code (**such as size, cyclomatic complexity**, etc.) and also reportcertain analytical conclusions.
- These also check the conformance of thecode with the prescribed coding standards.
- A major practical limitation of the static analysis tools lies in their inability to analyse run-time information such as dynamic memory references usingpointer variables and pointer arithmetic, etc.
- In a high level programminglanguages, pointer variables and dynamic memory allocation provide thecapability for dynamic memory references.
- However, dynamic memoryreferencing is a major source of programming errors in a program.
- Static analysis tools often summarise the results of analysis of everyfunction in a polar chart known as Kiviat Chart.
- A Kiviat Chart typically shows the analysed values for cyclomatic complexity, number of source lines,percentage of comment lines, Halstead's metrics, etc.

Static analysis testing technique:

Informal reviews

-In this technique, the team of reviewers just checks the documents and gives comments.

Formal reviews

-It is well structured and documented and follows six main steps: planning, kick-off, and preparation, review meeting, rework follow-up.

- **Technical reviews:**

❖ The team of technical experts will review the software for technical specification.

❖ The purpose is to pin out the difference between the required specification and product design and then correct the flaws.

❖ It focuses on technical documents such as test strategy, test plan andrequirement

specification documents.

- **Walk-through**

❖ The author explains the software to the team and teammates can raise questions if they have any.

❖ It is headed by the author and review comments are noted down.

- **Inspection process**

❖ The meeting is headed by a trained moderator. A formal review is done, a record is maintained for all the errors and the authors are informed to make rectifications on the given feedbacks.

- **Static code review**

❖ Code is reviewed without execution; it is checked for syntax, coding standards and code optimization. It is also referred to as white box testing.

Advantages:

1) It is a fast and easy technique used to fix errors.
2) It helps in identifying flaws in code.
3) With the help of automated tools it becomes very easy and convenient to scan and review the software.
4) With static testing it is possible to find errors at an early stage of the development life cycle.

Disadvantages:

1) It takes a lot of time to conduct the testing procedure if done manually.
2) Automated tools work for a restricted set of programming languages.
3) The automated tools simply scan the code and cannot test the code deeply.

4.9.2 Dynamic Analysis Tools

- Dynamic program analysis tools can be used to evaluate several programcharacteristics based on an analysis of the run time behavior of a program.
- These tools usually record and analyses the actual behavior of a programwhile it is being executed.
- A dynamic program analysis tool (also called dynamic analyzer) usually collects execution trace information byinstrumenting the code.
- Code instrumentation is usually achieved by insertingadditional statements to print the values of certain variables into a file tocollect the execution trace of the program.
- The instrumented code when executed, records the behavior of the software for different test cases.
- An important characteristic of a test suite that is computed by a dynamic analysis toolis the extent of coverage achieved by the test suite.
- After software has been tested with its full test suite and its behavior recorded, the dynamic analysis tool carries out a post execution analysis andproduces reports which

describe the coverage that has been achieved by thecomplete test suite for the program.

- For example, the dynamic analysis toolcan report the statement, branch, and path coverage achieved by a testsuite.
- If the coverage achieved is not satisfactory more test cases can bedesigned, added to the test suite, and run. Further, dynamic analysis resultscan help eliminate redundant test cases from a test suite.
- Normally the dynamic analysis results are reported in the form of a histogram or pie chart to describe the structural coverage achieved for different modules of the program.
- The output of a dynamic analysis tool canbe stored and printed easily to provide evidence that thorough testing hasbeen carried out.

Dynamic testing techniques

- Unit testing

-As the name suggests individual units or modules are tested. The source code is tested by the developers

- Integration testing

-Individual modules are clubbed and tested by the developers. It is performed in order to ensure that modules are working in the right manner and will continue to perform flawlessly even after integration.

- System testing

  -it is performed on a complete system to ensure that the application is designed according to the requirement specification document.

Advantages

1)it identifies weak area in a runtime.
2)it helps in performing detailed analysis of code.
3)it can be applied with any application.

Disadvantages

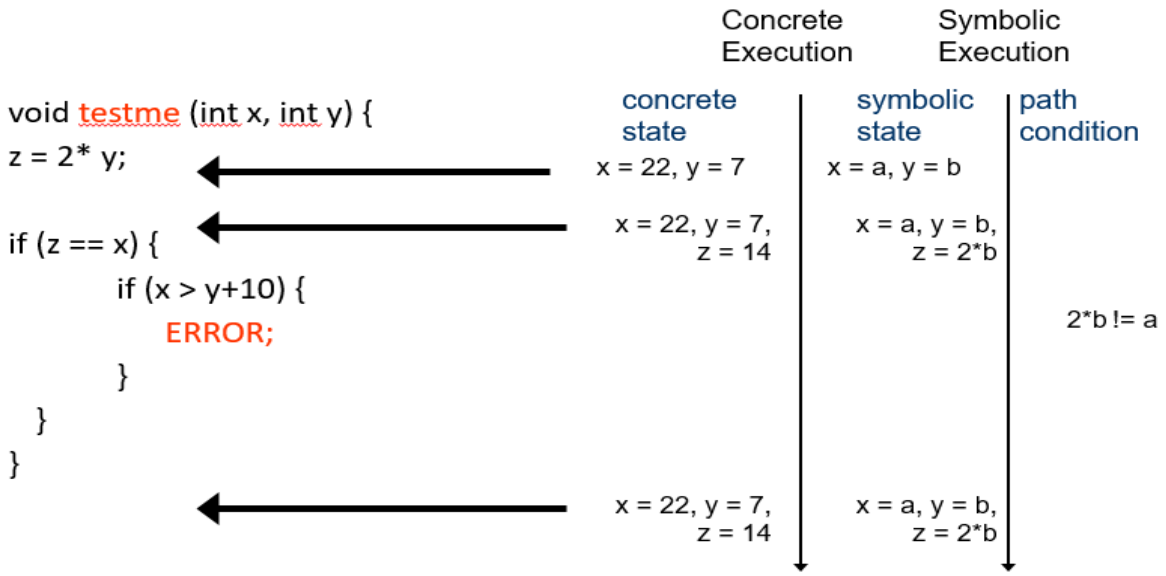
1)it is not easy to find a trained software tester to perform dynamic testing.
2)it becomes costly to fix errors in dynamic testing.

sno	Static analysis	Dynamic analysis
1	Static analysis is a testing process that is done at the early stages of the development life cycle.	Dynamic analysis is a testing process done later stages of the development life cycle.
2	It involves a walkthrough and code review.	It involves functional and non-functional testing.
3	This testing is a verification process.	This testing is a validation process.
4	This type of testing is done before the execution of code.	This type of testing is done during the execution of code.
5	It is about prevention.	It is about a cure.
6	It is cost-effective.	It is costly.

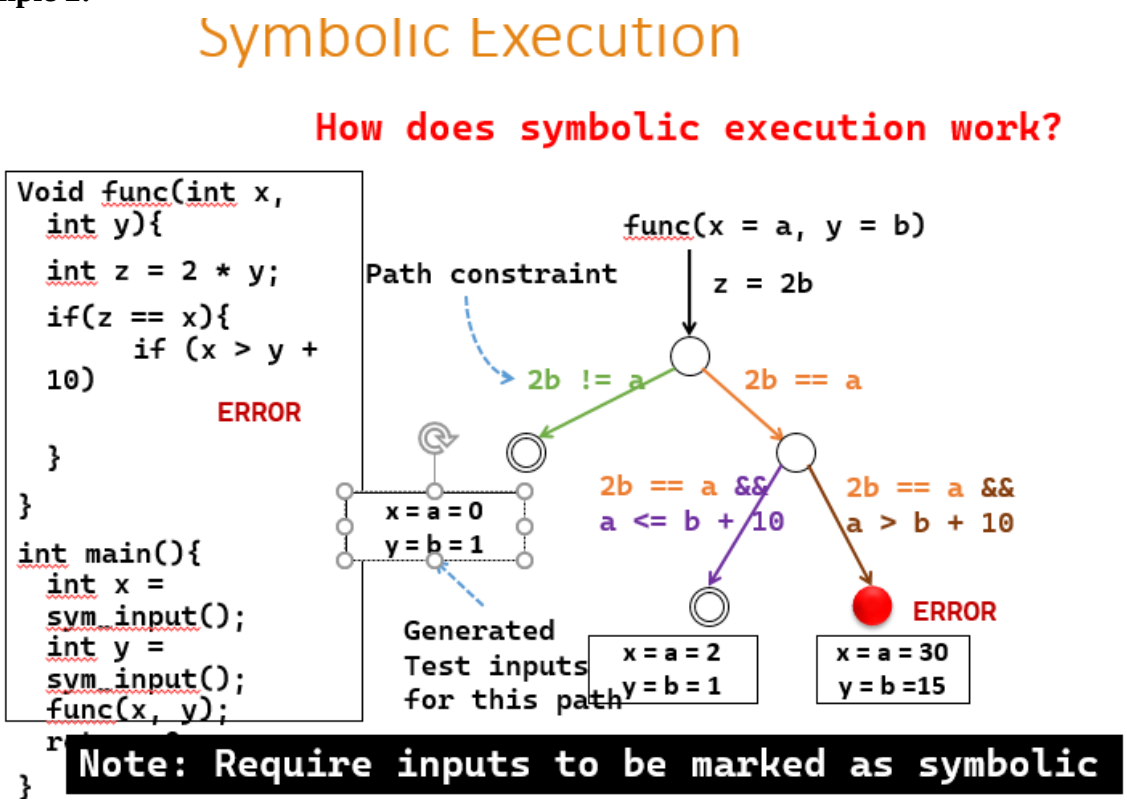
4.10Symbolic Execution

- Symbolic execution is a testing technique in which we have use symbols to see what the output of the code is without actually running the code.
- This helps to find and fix the errors in the software without doing a real test.
- Symbolic execution is a technique for testing software by symbolically evaluating the program's control flow and data flow.
- It is a type of dynamic analysis, which means that it tests the program by executing it with symbolic input values instead of concrete input values.
- Symbolic execution involves analyzing the code without running it with actual values.
- Execute the program with symbolic valued inputs (**Goal: good path coverage**)
- Represents *equivalence class of inputs* with first order logic formulas (**path constraints**)
- One path constraint abstractly represents all inputs that induces the program execution to go down a specific path
- Solve the path constraint to obtain one representative input that exercises the program to go down that specific path
- **Symbolic execution implementations**: KLEE, Java PathFinder, etc.
- Instead of concrete state, the program maintains **symbolic states**, each of which maps variables to symbolic values
- **Path condition** is a quantifier-free formula over the symbolic inputs that encodes all branch decisions taken so far
- All paths in the program form its **execution tree**, in which some paths are feasible and some are infeasible

Example1-



Example 2:



Example3-

```
function divide(int a, int b)
{
int (b!=0)
result =a/b;
else
result=-1
return result;
}
```

- In above code, the result is a symbolic variable we can analyze the code behavior in both cases without having specific values for ‘a’ and ‘b’ at this stage. This helps us understand how the code will react to various scenarios.
- Symbolic execution is often used in more complex programs to find the major bugs.
- This technique also helps us to understand how the code responds to different inputs without running it with actual data.
- It is an impotent technique for debugging and testing software to identify possible problems and ensure robustness.

Steps to use symbolic Execution:

- The execution requires a selection of paths that are exercised by a set of data values. A program, which is executed using actual data, results in the output of a series of values.
- In symbolic execution, the data is replaced by symbolic values with set of expressions, one expression per output variable.
- The common approach for symbolic execution is to perform an analysis of the program, resulting in the creation of a flow graph.
- The flow graph identifies the decision points and the assignments associated with each flow. By traversing the flow graph from an entry point, a list of assignment statements and branch predicates is produced.

Issues with Symbolic Execution:

- Symbolic execution cannot proceed if the number of iterations in the loop is known.
- The second issue is the invocation of any out-of-line code or module calls.
- Symbolic execution cannot be used with arrays.
- The symbolic execution cannot identify of infeasible paths.

Symbolic Execution Application:

- Path domain checking
- Test Data generation
- Partition analysis
- Symbolic debugging

4.11.Model checking

- Model checking is technique for verifying that a software system satisfies the desired properties.
- In this technique, a model of the system is created and then using a model checker. We exhaustively explore all possible states of the model to check the properties of the system.
- Model checking can be used in both software testing and debugging. In software testing, it can be used to verify that the system meets its requirements. In software debugging, it can be used to identify the root cause of a bug.
- Once the model and the desired properties have been specified, a model checker can be used to verify that the model satisfies the properties.

For example- ATM system

- ✓ Consider a ATM system in which we have to design a function for withdrawal of money say withdraw().
- ✓ This function allows user to withdraw money from his/her account. In order to implement this function, following requirements need to be satisfied.

1. The user must have a positive balance in their account in order to withdraw money.
2. The user can not withdraw more money than they have in their account.
3. The user's balance must be updated after each withdrawal

- ✓ Before actually executing the code, we can use model checking technique to verify that the withdraw() function meets these requirements.
- ✓ In this technique, we first create a model of the withdraw() function.
- ✓ We must specify the above requirements as properties of that model.
- ✓ The model checker would explore all possible states of the function to verify that the function satisfies all these properties.
- ✓ For instance, if there is a negative balance, then there must not be a path to 'withdraw amount' state.
- ✓ Similarly if the balance amount greater than the individual amount then only there should be a path to 'withdraw amount' state.
- ✓ If the model checker finds a state in which the function does not update the user's balance after a withdrawal then it should report a bug.
- ✓ By identifying this issue in our model, we can locate the corresponding code in our software and correct the logic to prevent this scenario from happening.
- ✓ Model checking is particularly valuable for complex systems, where manually testing all possible combinations and path would be impractical.